

To update store state within a utility function in Next.js, the method depends on whether you are using **Zustand** or **Redux**. Because utility functions typically exist outside the React component lifecycle, you cannot use hooks like `useStore` or `useDispatch` directly.  Next.js +2

For Zustand

Zustand is highly compatible with utility functions because the store object itself exposes methods to update state outside of React.  YouTube +1

- **Direct Store Access:** Import your created store (e.g., `useStore`) and call its `getState()` or `setState()` methods.
- **Example:**

javascript

```
import { useStore } from './store';

export const myUtilityFunction = (newData) => {
  // Access and update state directly without hooks
  useStore.setState({ data: newData });
};
```

- **Documentation:** Review the Zustand Guide on Setup with Next.js for advanced patterns.  JavaScript in Plain English +4

For Redux Toolkit

Redux generally requires a `dispatch` function to update state.  YouTube +2

- **Pass Dispatch as an Argument:** The most reliable way is to pass the `dispatch` function from a component into your utility function.
- **Direct Store Import (Client-Side Only):** You can import the singleton `store` object directly into your utility file and call `store.dispatch()`, though this is often discouraged in Next.js to avoid hydration mismatches during SSR.
- **Example (Passing Dispatch):**

javascript

```
// Utility file
export const updateMyState = (dispatch, value) => {
  dispatch(myAction(value));
};
```

```
// Component file
const dispatch = useDispatch();
updateMyState(dispatch, 'new value');
```

- **Official Resource:** Refer to the Redux Toolkit Next.js Setup for managing store instances per request.  Telerik.com +4

Important Constraints

- **Client vs. Server:** Store updates via utility functions are strictly **client-side** operations. You cannot update a client-side store (like Zustand or Redux) from a Server Component or a [Next.js Server Action](#).
- **Hydration:** If your utility function runs immediately on page load, ensure it does not cause a mismatch between server-rendered HTML and client-side state.  Stack Overflow +3